

Basic Git

Basic Git

- Introduction to Git
- Git lifecycle
- Common Git command
- Git Workflow
- Working with Remote Repository
- Version controlling using Git

GitHub

- GitHub is a cloud-based platform where you can store, share, and work together with others to write code.
- Storing your code in a "repository" on GitHub allows you to:
 - Showcase or share your work.
 - Track and manage changes to your code over time.
 - Let others review your code, and make suggestions to improve it.
 - Collaborate on a shared project, without worrying that your changes will impact the work of your collaborators before you're ready to integrate them.
- Collaborative working, one of GitHub's fundamental features, is made possible by the open-source software, Git, upon which GitHub is built.

Version Control System

- A version control system, or VCS, tracks the history of changes as people and teams collaborate on projects together.
- As developers make changes to the project, any earlier version of the project can be recovered at any time.
- Developers can review project history to find out:
 - Which changes were made?
 - Who made the changes?
 - When were the changes made?
 - Why were changes needed?

- VCSs give each contributor a unified and consistent view of a project, surfacing work that's already in progress.
- Seeing a transparent history of changes, who made them, and how they contribute to the development of a project helps team members stay aligned while working independently.
- In a distributed version control system, every developer has a full copy of the project and project history.
- Unlike once popular centralized version control systems, DVCSs don't need a constant connection to a central repository.

Introduction to Git

- Git is the most popular distributed version control system that intelligently tracks changes in files.
- Git is commonly used for both open source and commercial software development, with significant benefits for individuals, teams and businesses.
- Git is particularly useful when you and a group of people are all making changes to the same files at the same time.
- Git lets developers see the entire timeline of their changes, decisions, and progression of any project in one place. From the moment they access the history of a project, the developer has all the context they need to understand it and start contributing.\
- Developers work in every time zone. With a DVCS like Git, collaboration can happen any time while maintaining source code integrity. Using branches, developers can safely propose changes to production code.
- Businesses using Git can break down communication barriers between teams and keep them focused on doing their best work. Plus, Git makes it possible to align experts across a business to collaborate on major projects.

How do Git and GitHub work together?

Feature	Git	GitHub
Type	Version Control System (software)	Cloud Hosting Service
Works	Locally on your computer	Online (cloud)
Internet Required	✗ No	✓ Yes
Main Function	Track and manage code changes	Share and collaborate on repositories
Creator	Linus Torvalds	Microsoft (acquired from founders)
Commands	<code>git add, git commit, git push</code>	No direct commands — web interface/API
Example Use	Save versions of your code	Upload code and work with teams
Storage Location	Local machine	Remote server (GitHub cloud)

Signing up for a new personal account

- Navigate to <https://github.com/>.
- Click Sign up.
- Follow the prompts to create your personal account.
- During sign up, you'll be asked to verify your email address.
Without a verified email address, you won't be able to complete some basic GitHub tasks, such as creating a repository.
- Now that you've created your personal account, we'll start to explore the basics of GitHub

Exercise : GitHub Essentials

- Lets perform basic GitHub essentials like repositories, branches, commits, and pull requests.
 - Step 1: Create and use a repository
 - Step 2: Create and manage a branch
 - Step 3: Make changes to a file and push them to GitHub as commits (commit changes)
 - Step 4: Open a pull request
 - Step 5: Merge your pull request

Branch	A separate workspace for changes	Like a rough copy of your notebook
Clone	Download project to your computer	Copying a GitHub project to edit locally
Fork	Your own copy of someone else's repo	Making a personal duplicate online
Merge	Combine branches together	Mixing your notes back into the main notebook
Pull Request	Request to add your changes	Asking to merge your work into main
Remote	Online version of your project	Cloud storage for your code
Upstream	Original main repository	The main source you copied from

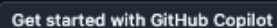
Git/GitHub Term	Simple Meaning	Cricket Example 🏏
Branch	A separate line of work	Like a batsman practicing a new batting style in the nets without affecting the main match. Once perfected, it's added to the main team strategy.
Clone	Download a copy of a project	Like taking a video of a cricket match to your home computer so you can study it and practice later.
Fork	Your own copy of someone's project on GitHub	Like creating your own cricket team by copying another team's strategy. You can train differently and make your own changes.
Merge	Combine one branch's work into another	Like when the coach combines all player strategies (batting, bowling, fielding plans) into one final team strategy.
Pull Request (PR)	Request to merge your changes	Like a player asking the coach to include his new shot or bowling style into the main game plan — "Please review and add my improvement."
Remote	The online version of your repository	Like the official cricket board's storage where the final match plans and videos are saved (accessible to the team).
Upstream	The original main repository	Like the national cricket team's official playbook , and your team is a local copy (downstream) learning from it.

Step 1: Create and use a repository

- You can think of a repository as a folder that contains related items, such as files, images, videos, or even other folders.
- A repository usually groups together items that belong to the same "project" or thing you're working on.
- Often, repositories include a README file, a file with information about your project.
- README files are written in Markdown, which is an easy-to-read, easy-to-write language for formatting plain text.
- GitHub lets you add a README file at the same time you create your new repository.
- GitHub also offers other common options such as a license file, but you do not have to select any of them now.

Steps to create a repository

- In the upper-right corner of any page, select , then click New repository.
- In the "Repository name" box, type "any name ".
- In the "Description" box, type a short description. For example, type "This repository is for practicing the GitHub Flow."
- Select whether your repository will be Public or Private.
- Select Add a README file.
- Click Create repository.



```
echo "# dummy" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/Divyathakur273/dummy.git
git push -u origin main
```



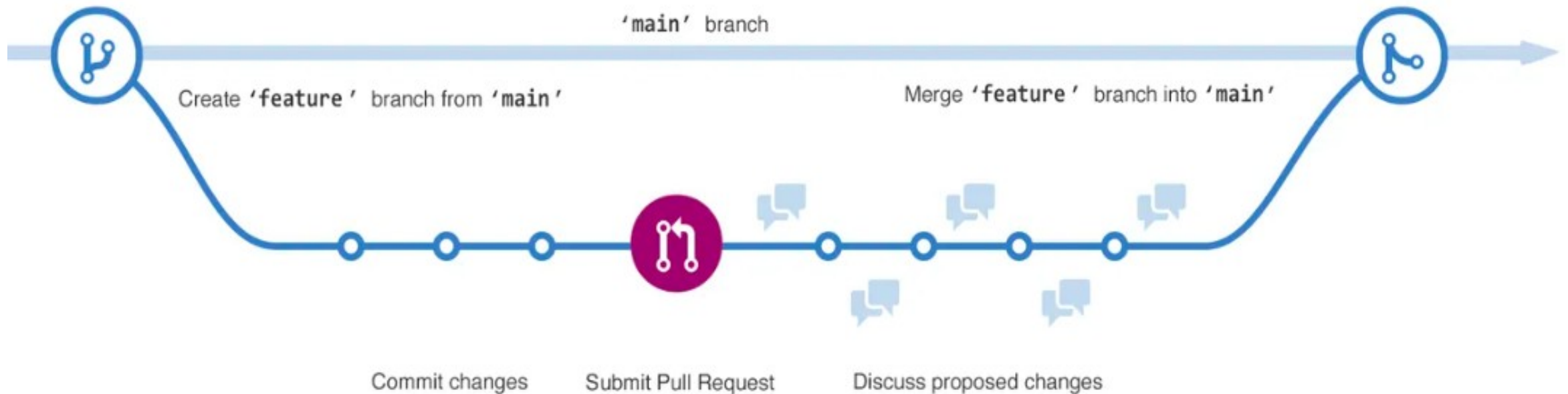
```
git remote add origin https://github.com/Divyathakur273/dummy.git
git branch -M main
git push -u origin main
```



Step 2: Create a branch

- Branching lets you have different versions of a repository at one time.
- By default, your repository has one branch named main that is considered to be the definitive branch.
- You can create additional branches off of main in your repository.
- Branching is helpful when you want to add new features to a project without changing the main source of code.
- The work done on different branches will not show up on the main branch until you merge it, which we will cover later in this guide.
- You can use branches to experiment and make edits before committing them to main.
- When you create a branch off the main branch, you're making a copy, or snapshot, of main as it was at that point in time. If someone else made changes to the main branch while you were working on your branch, you could pull in those updates.

- This diagram shows:
 1. The main branch
 2. A new branch called feature
 3. The journey that feature takes before it's merged into main



Creating a branch

- Click the Code tab of your repository.
- Above the file list, click the dropdown menu that says main.
- Type a branch name, “new”, into the text box.
- Click Create branch: “new” from main.
- Now you have two branches, main and new.
- Right now, they look exactly the same.
- Next you'll add changes to the “new” branch.

Step 3: Make and commit changes

- When you created a new branch in the previous step, GitHub brought you to the code page for your new branch, which is a copy of main.
- You can make and save changes to the files in your repository.
- On GitHub, saved changes are called commits.
- Each commit has an associated commit message, which is a description explaining why a particular change was made.
- Commit messages capture the history of your changes so that other contributors can understand what you've done and why.

Under the new branch you created, click the README.md file.

- To edit the file, click .
- In the editor, write a bit about yourself.
- Click Commit changes.
- In the "Commit changes" box, write a commit message that describes your changes.
- Click Commit changes.
- These changes will be made only to the README file on your new branch, so now this branch contains content that's different from main.

Step 4: Open a pull request

- Now that you have changes in a branch off of main, you can open a pull request.
- Pull requests are the heart of collaboration on GitHub.
- When you open a pull request, you're proposing your changes and requesting that someone review and pull in your contribution and merge them into their branch.
- Pull requests show diffs, or differences, of the content from both branches. The changes, additions, and subtractions are shown in different colors.
- As soon as you make a commit, you can open a pull request and start a discussion, even before the code is finished.
- In this step, you'll open a pull request in your own repository and then merge it yourself. It's a great way to practise the GitHub flow before working on larger projects.

- Click the Pull requests tab of your repository.
- Click New pull request.
- In the Example Comparisons box, select the branch you made, new, to compare with main (the original).
- Look over your changes in the diffs on the Compare page, make sure they're what you want to submit.
- Click Create pull request.
- Give your pull request a title and write a brief description of your changes. You can include emojis and drag and drop images and gifs.
- Click Create pull request.

Step 5: Merge your pull request

- In this final step, you will merge your new branch into the main branch.
- After you merge your pull request, the changes on your new branch will be incorporated into main.
- Sometimes, a pull request may introduce changes to code that conflict with the existing code on main.
- If there are any conflicts, GitHub will alert you about the conflicting code and prevent merging until the conflicts are resolved.
- You can make a commit that resolves the conflicts or use comments in the pull request to discuss the conflicts with your team members.

- Merge your branch into the main branch.
- At the bottom of the pull request, click Merge pull request to merge the changes into main.
- Click Confirm merge. You will receive a message that the request was successfully merged and the request was closed.
- Click Delete branch. Now that your pull request is merged and your changes are on main, you can safely delete the new branch.
- If you want to make more changes to your project, you can always create a new branch and repeat this process.
- Click back to the Code tab of your repository to see your published changes on main.

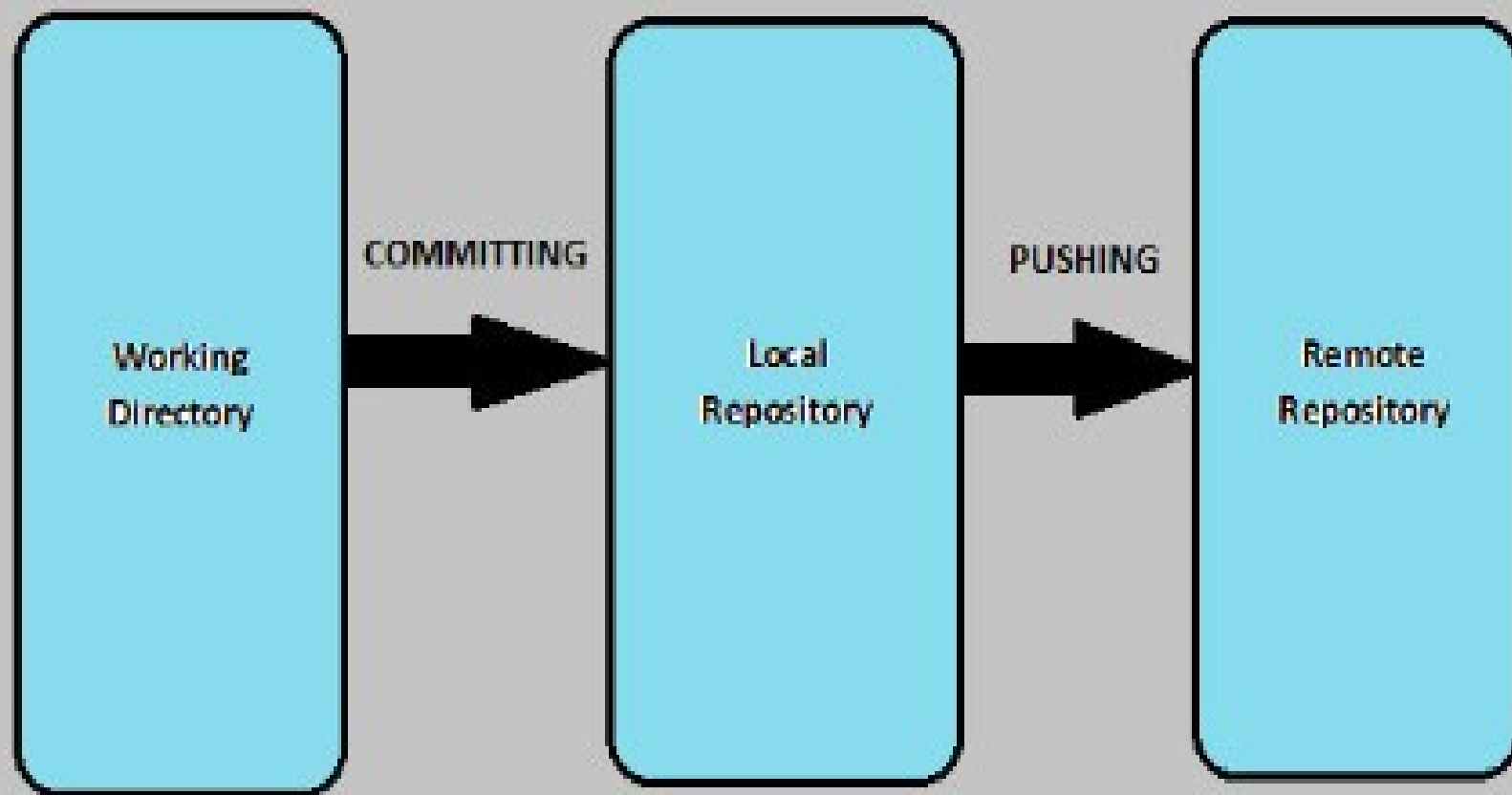
Other Platforms

- GitHub is an open source Git hosting platform that is available as a cloud service and as a self-hosted installation.
- Similarly , we have other popular platforms as Bitbucket and gitlab , which are also open source Git hosting platform.
- But the most popular platform is Github with many good features than others .

Difference between GIT and GitHub

- **Git**: It is a Distributed Version Control System for tracking versions of files.
- **Github**: It is web portal and cloud hosting service for your Git repositories.
- **Git** uses commands to track versions of your files.
- **Github** is just a remote platform where these files are hosted.
- **Git** allows you to track versions of your code in your local machine.
- **Github** :if you want a remote backup of your code or want to publish your code to a community then you've to push it to Github.

- It's not mandatory to use Github.
- If you're the sole person working on the project and don't need to publish your code to the world then you can simply avoid using Github.
- Just track the versions of your code in local repository of your machine using Git.
- By this point, it'd be clear that committing the code using Git in your local machine doesn't automatically upload it to Github too.
- There are two distinct terms for these activities - Committing and Pushing.
- Committing is capturing the changes from your working copy to your local repository. On the other hand, Pushing is uploading the captured changes from your local repository to Github.



The Git Lifecycle

- The lifecycle of Git consists of four main stages:
- Create
- Modify
- Stage
- Commit



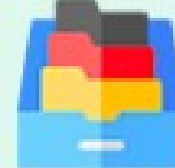
Working
Directory

git init



Initialization

git add

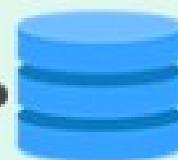


Staging Area



github

git push

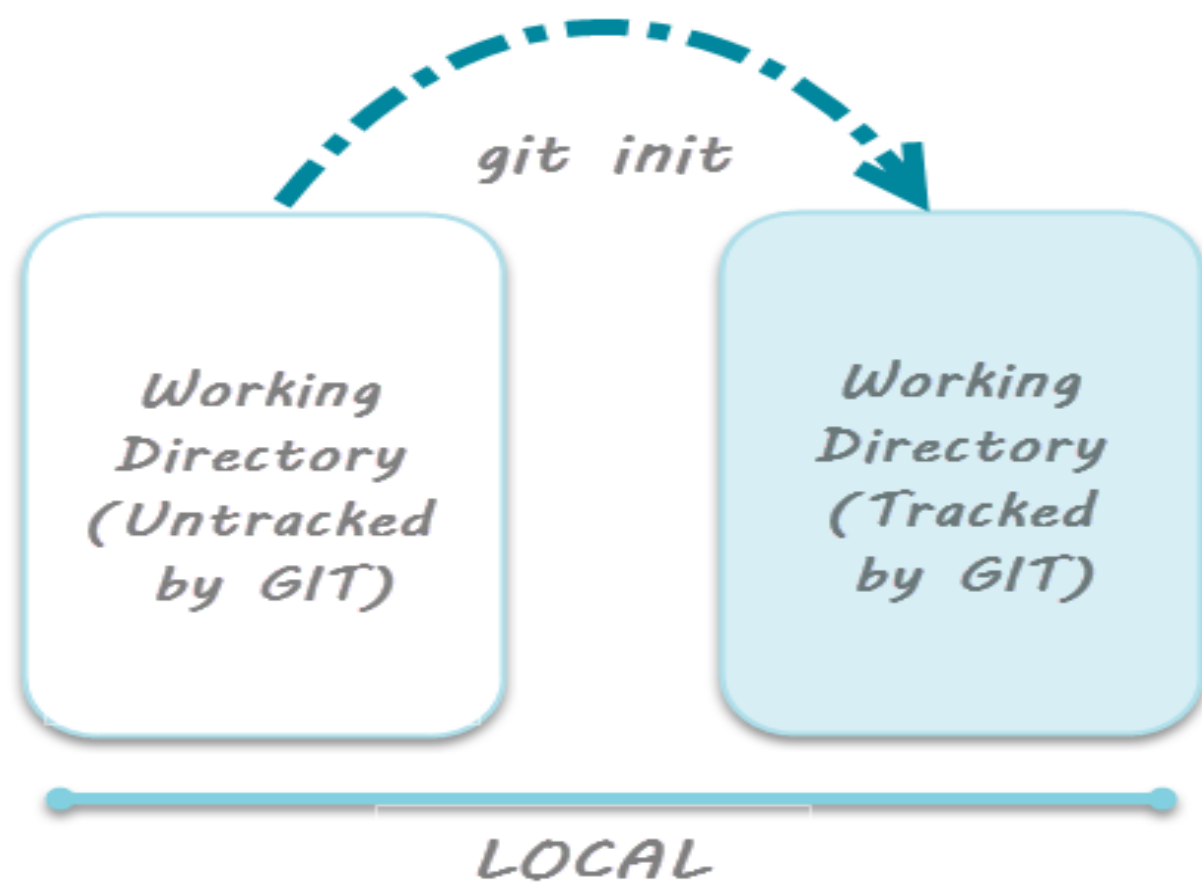


Local
Repository

git commit

Create

- The first stage in the lifecycle of Git is to create a new repository.
- You can think of it just like a project folder.
- This can be done either locally on your computer or on a remote server.
- To create a new repository, you can use the command "git init" followed by the name of the repository.



Modify

The next stage in the lifecycle of Git is to modify the files in the repository.

This can be done using any text editor or integrated development environment (IDE) of your choice.

When you make changes to a file, Git automatically detects those changes and marks the files as "modified."

Stage

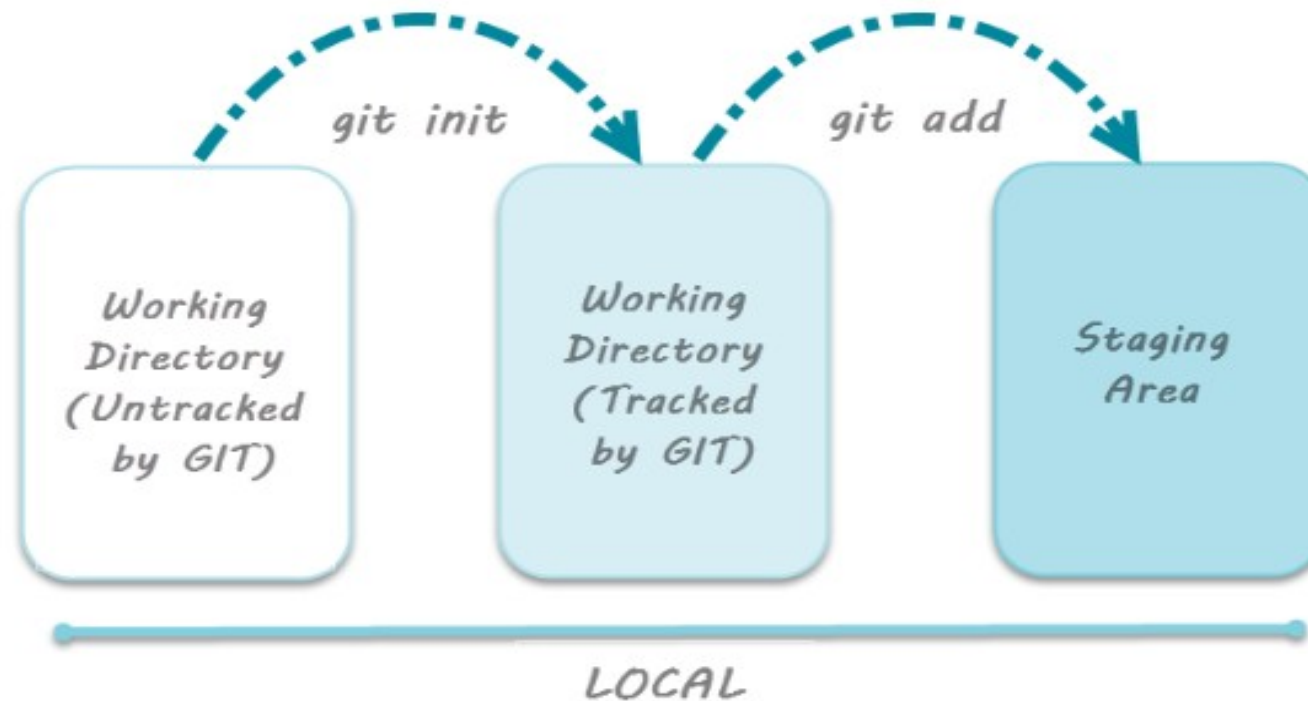
The third stage in the lifecycle of Git is to stage the changes you have made.

Staging is the process of preparing your changes to be committed to the repository.

You can stage your changes using the "git add" command followed by the name of the file you want to stage.

You can also use the "git add" command to stage all changes in the repository.

- Staging area is the playground where you group, add and organize the files to be committed to Git for tracking their versions.
- It's important to make a quick note of the term called indexing here. Indexing is the process of adding files to the staging area. In other words, index constitutes of files added to the staging area.



Commit

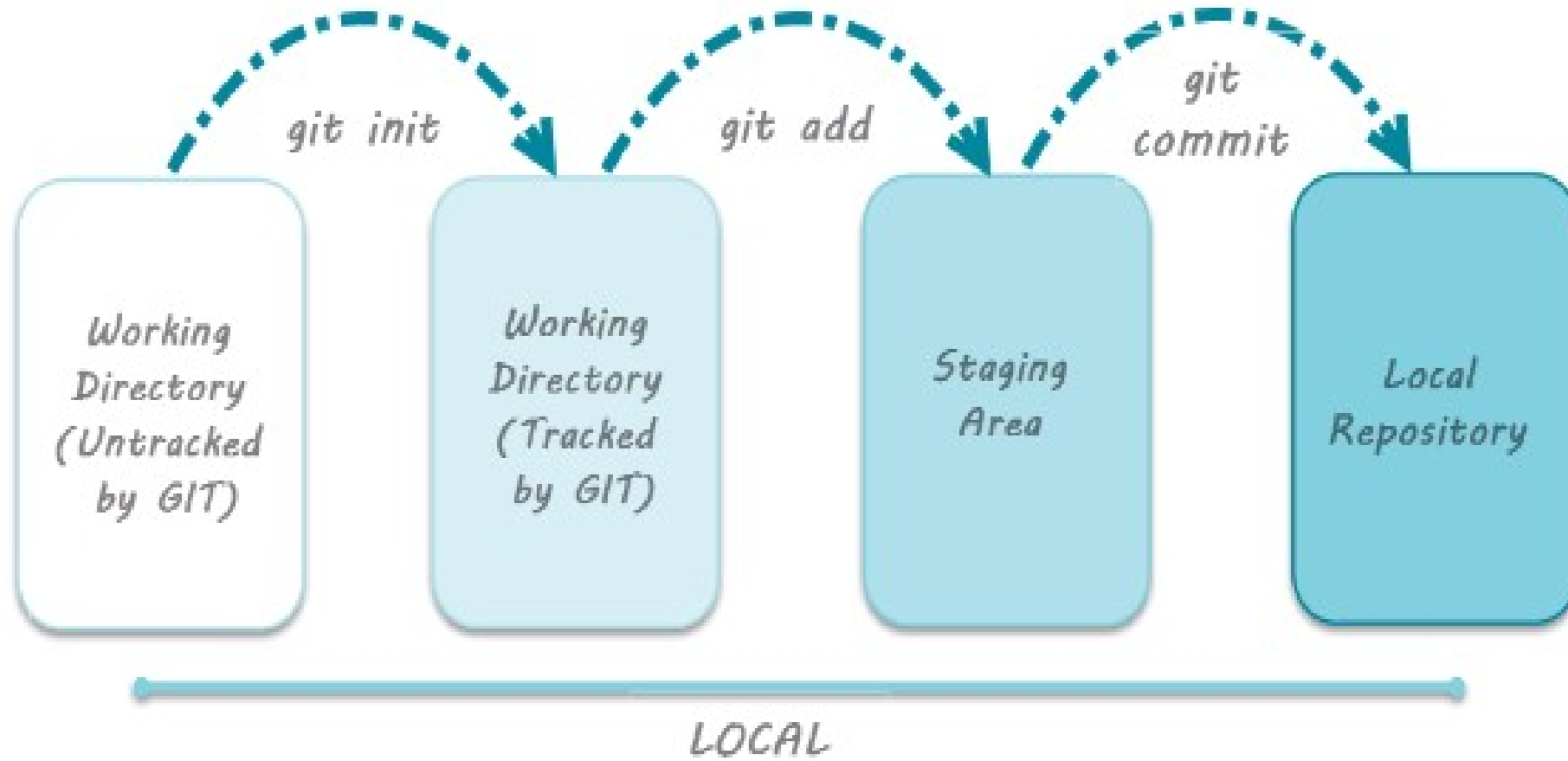
The final stage in the lifecycle of Git is to commit your changes to the repository.

A commit is a permanent snapshot of the changes you have made to the repository, and by default, it goes to the master branch.

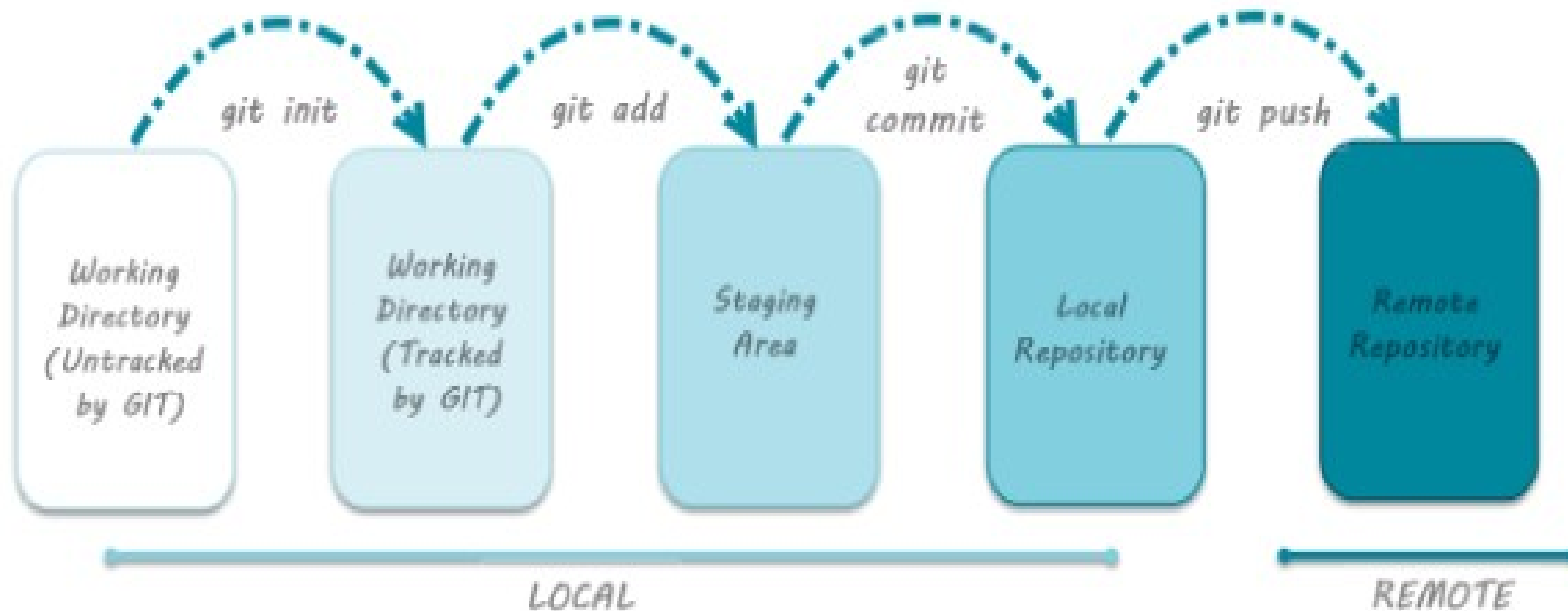
When you commit your changes, Git creates a new commit object that contains the changes you have made, along with a message describing what changes were made.

To commit your changes, you can use the "git commit" command followed by a message describing the changes you have made.

Note: `git commit -m"your message"` - Command to commit files to Git repository with message.



- Once you have committed your changes, you can push those changes to a remote repository using the "git push" command. This will upload your changes to the remote repository, making them available for other developers to pull down and work with.



Need of Staging Area

- **Faster Git operations:**
 - staging helps in recording the changes even before committing them to Git database.
- **Visualizing the commit before actual commit :**
 - staging area is the state in which the files reside before they're committed.
 - That is, staging area actually lets you visualize the group of changes that will be recorded by Git.
 - Essentially this gives you fine-grained control over what gets committed to Git and what does not.
- **Splitting work into separate related commits:**
 - suppose you're working on a new feature whose timeline of development runs into several days. So, while you're working on the feature you are required to refactor a small part of the code.
 - In this case, you can quickly make the required changes, stage the required file and resume work on the feature.
 - All of the changes i.e. feature changes and refactor code can be committed at once from the staging area.

GIT

- Git can be used through two different methods with three different sub-methods. Both ways have their own advantages and drawbacks. Git can be used via:
- Shell
- GUI software

Git through Shell

- Shell as you must have already known is used for accessing the operating system services.
- Shell is used through a command-line interface.
- Git can be used in two different ways on any operating system through shell.
- These are:
 - Git Bash
 - Git CMD

Git Bash

- When Git was developed it was coded in a similar way to use it on a command line.
- Originally developed for the Linux Kernel project, the idea was the same originally.
- Bash as you know is the terminal of any Linux OS.
- Git Bash is the bash that runs the Git commands on it.
- It is similar to the Unix shell with the same commands as of Linux.
- You can operate git from Git Bash on any operating system.

Git CMD

- Git CMD is used on Windows.
- If you are familiar with the command prompt i.e. the shell used on Windows then you can use this one to run Git.
- Git CMD is nothing but CMD which can run Git commands.
- Although you can use it, the most recommended method is the Git bash.
- The reason being CMD is confined to Windows only. It will confine your knowledge to Windows only.

Git through GUI

- Git GUI is using Git with the help of a graphical interface.
- This will diminish your shell usage and everything for which you were writing a command, you will now press some button or do it through the mouse.
- Using Git on a shell is always preferred among the developer community.
- Also, the big tech giants like Google, Amazon, etc uses Git on the command line because of its advantages and security.
- Advantages including that there are some features on a shell that are not available to GUI like ssh tunnel connecting to the remote server etc.
- Also, command line will let you automate your Git work.
- Some features are first released to command line also before the GUI and some of them don't even release to GUI at all.
- All in all, GIT on a shell is more powerful than on the GUI.

Git Download

- **Git Bash**

- It allows you to execute Git commands in conjunction with Unix commands in a Windows environment.
- This tool will be downloaded automatically when you install GIT.
- Download the git by going to the following link :

<https://git-scm.com/downloads>

- **Git cmd**

- It allows you to execute Git commands in a traditional Windows Command Prompt type environment.
- You also get the option of selecting Git CMD while installing Git.

Git Commands

- **git help:**
 - This command will display all the information about Git.
- If we need to see all the available commands in Git, we need to use the following command.
 - **git help -a or git help --a or git help --all**
- **git help -g** : This command gives common git guides.
- **git help <guide_name>** : This command gives manual page of specific guide.
- **git help <command name>** : This command gives information about a specific command .

Set Up Default Credentials for Git Config

- `vi ~/.gitconfig` : Command to check the personal configuration settings.
- As we install Git in our system, the configuration file takes default values for some of the fields.
- This creates a conflict in the identity of the programmer.
- We will Set Up Default Credentials for Git Config , so that no random credentials are set by default .
- You will not get any value if you have not already set it.

- **Set user's UserName in Git Config:**

- `git config --global user.name "Your UserName"`
- eg :
 - `git config --global user.name "Divya Thakur"`
- To confirm :
 - `git config --global user.name`

- **Set user's Email in Git Config:**

- `git config --global user.email "Your EmailID"`
- eg :
 - `git config --global user.email "divyathakurdt73@gmail.com"`
- To confirm :
 - `git config --global user.email`

- View the complete list of setting in Git Config:
 - `git config --list`
- The below command will be used to set a default editor for gitbash for eg notepad++ for windows .
 - `git config --global core.editor <directory address>`

How to create repository in cli

- `cd <path of the directory> :`
 - Changing the directory is important as you are always shuffling between different directories all the time while working with Git Bash.
 - Directory is a technical term for the folder.
- Create a New Directory:
 - `mkdir <folder name>`
- Create Bare Git Repository(empty directory with just a hidden .git folder):
 - Create a New Project/Folder (using `mkdir`)
 - Browse to New Project(using `cd`)
 - Initialize Git Repository for the Project : `git init`
 - `git init` is a standard GIT command and it initializes the directory with a .git folder used for tracking versions of project artifacts.

.git

- **ls -a** : command used to see the hidden files in a directory.
- **Contents of .git folder :**
- **Hooks:** This folder has a few script files inside it. These script files are known as Git Hook Scripts. Git hooks are the scripts that are executed before or after the events. These events can be any Git event including the common Git events like commit, push, or receive.
- **Info:** Info folder contains the exclude file inside it. As the name suggests, exclude file is used for excluding some specific patterns in the code that you don't want Git to read or execute. (.gitignore)
- **Objects**
- **Config :** The config file contains your configuration
- **Description:** Description contains the data about the repositories which can be seen on GitWeb only. It is a web-based visualizer. Git comes with a CGI script called GitWeb.
- **HEAD**

- **Objects:**

- Objects folder is a very important folder in the .git directory.
- In Git, everything is saved in the objects folder as a hash value.
- By everything I mean every commit, every tree or every file that you create is saved in this directory.
- With every object, there is a hash value linked to it, through which Git knows where is what.
- The folders are created accordingly.
- example :
 - 1.) create a file in repository .check the object folder .
 - 2.) Add some text to this file with command : `echo "this is my first file" > filename.txt`
 - 3.)Add your file to the staging area : `git add filename.txt`
 - 4.)Commit the changes : `git commit -m "Committing my Changes"`
- These contain the hash value of the events you just did. Git tracks and recognizes everything by converting it to the hash value. The folders are named according to the hash value saved inside.
- hashing is a popular method (data structure) of converting your data to hash value (random combinations of letters and digits) .
- Hashing in Git is used to be able to trust and create stable data where collision does not occur which means two same files can be saved because they will have different hash values

- Head: Head file contains the reference to the branch we are currently working on.
- It is a symbolic reference to the branch and not the normal reference.
- The difference being that the normal reference contains the Hash value as we saw in the above Objects folder.
- It refers to something directly like pointers in programming.
- To see the content of the Head file you can just type the command:

cat .git/HEAD

Deleting .git Folder

- Deleting this folder will delete our entire history.
- If you haven't cloned it, all the version and history is erased from Git.
- Though your source file will remain as it is.
- To delete the .git folder, just go to Git Bash and delete the folder by typing the command:

```
rm -rf .git
```

Git Staging

- Git Add Command
- Git Status Command
- Git Remove Command
- Lets discuss them one by one in detail.

Git Status Command

- It returns information about the current state of the repository.
- For e.g. a list of files changed, list of tracked changes on staging, untracked changes on local, and information about current branch & commits.
- **Staging Area:** Git has the concept of a staging area, where all the updates/changes are tracked. This holds the changes which you would like to commit. Changes can be a single line, files, or parts of files.
- **Untracked Changes:** Changes/files on the local directory and which are not pushed to staging are known as untracked changes.
- **Tracked Changes:** Changes/files on staging are known as tracked changes.
- Type **git status** in the command prompt. This will now list which files are staged, unstaged, and untracked.

Git add

- The **git add** command adds a change in the working directory to the Staging Area.
- It tells Git that there are few updates in the project, which the user wants to commit next.
- The thing here to note is that git add doesn't affect the remote repository, as changes are not actually recorded until you perform git commit.
 - Create a file inside repo using : touch filename.txt
 - enter data in the file : echo "this is my first file">filename.txt
 - Check the data in the file : cat filename.txt
 - write " git add " command in cli
 - see git status using "git status " command .
- O/P :
 - **Changes to be committed:** This shows the information of tracked files kept on staging. This is the one, which we added using git add command.
 - **Untracked files:** This shows the information of untracked files. These are the ones, which brought to the project earlier but still not pushed to staging.

- To add multiple file to staging area:
 - `git add <filename> <filename>`
- To add all file to staging area at once :
 - `git add *`

Git remove

- **git rm:**
 - The `git rm` command removes tracked changes from the Staging Area.
 - It tells Git that the updates which were pushed to staging earlier with `git add` command, are not ready to commit
 - **`git rm --cached <file>`** : command used to remove file .
 - To remove multiple file : **`git rm --cached <file> <file>`** .

O/P parameters of git status

- **branch master:** This says that the working tree is a master branch. **no**
- **commits yet:** This gives information on the commits, as of now there are no commits yet.
- **Untracked files:** This says that Git sees the files but has not started tracking changes yet and marked these as red.
- **Status messages:** This gives relevant instructions for staging/unstaging files.

Git Commit

- Committing the changes in Git:

git commit

- Committing the changes in Git without commit message:

git commit -m “this is my first commit”

Git does not recommend to commit without any message.

Git commit messages are necessary to look back and see the changes made during a particular commit.

If everyone will just commit without any message, no one would ever know what changes a developer has done.

- Committing without a message :

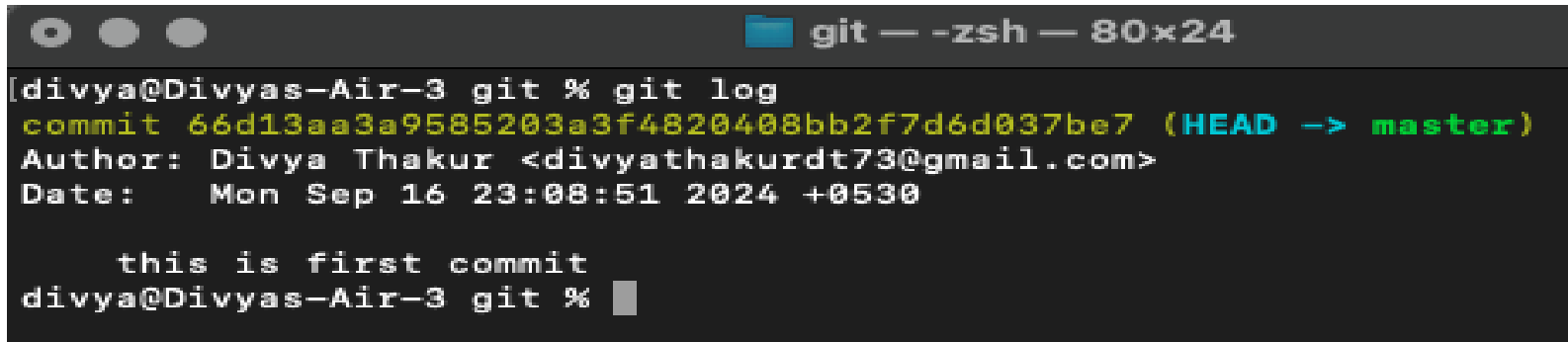
git commit -a --allow-empty-message -m "message"

Rename a file in git :

- `git mv <original_file_name> <new_file_name>`

Inspect & Track Changes

- git status : command used to inspect changes made
- git log :
 - Git log is a command used in Git to access the history of commits that the repository has gone through.
 - It captures all the changes happening inside the repository
- Output :

A terminal window titled 'git - zsh - 80x24' showing the output of the 'git log' command. The output displays a single commit with its hash, author information, date, and commit message.

```
[divya@Divyas-Air-3 git % git log
commit 66d13aa3a9585203a3f4820408bb2f7d6d037be7 (HEAD -> master)
Author: Divya Thakur <divyathakurdt73@gmail.com>
Date:   Mon Sep 16 23:08:51 2024 +0530

    this is first commit
divya@Divyas-Air-3 git %
```

- Commit Hash: The first part of the commit log is "commit hash" which is the hash value by which Git saves or refers everything internally.
- Commit Author: This part tells you about who committed the changes in the repository i.e. the author name.
- Commit Date: This part shows you the date and time of the commit with the day and time zone. It is important to have a day and time zone because many people work on a single project from different geographical locations.
- Commit Message: This part shows the commit message that was written while committing the changes.

- `git log --oneline`:
- As you can see some of the details that were previously visible are now hidden and the view is of only one-liner.

```
[divya@Divyas-Air-3 git % git log --oneline
66d13aa (HEAD -> master) this is first commit
divya@Divyas-Air-3 git %
```

- Commit Id: Every commit has a commit id which is actually a compressed version of commit hash. Since a commit hash is not a sequential number, compressing it still shows unique commits.
- Commit Message: commit message to understand what the commit was about.

- **git log <commit hash>**
- see the log details using the commit hash that we have got from **git log --oneline** command .

- Output:

```
[divya@Divyas-Air-3 git % git log 66d13aa
commit 66d13aa3a9585203a3f4820408bb2f7d6d037be7 (HEAD -> master)
Author: Divya Thakur <divyathakurdt73@gmail.com>
Date:   Mon Sep 16 23:08:51 2024 +0530

    this is first commit
```

- **git log <filename>**: used to get logs using filename:

```
git — -zsh — 80x24
divya@Divyas-Air-3 git % git log one.txt
commit 66d13aa3a9585203a3f4820408bb2f7d6d037be7 (HEAD -> master)
Author: Divya Thakur <divyathakurdt73@gmail.com>
Date:   Mon Sep 16 23:08:51 2024 +0530

    this is first commit
divya@Divyas-Air-3 git %
```


View Commits History for a range:

git log <since commitId>..<until commitId> :

- Since means, the last commit up to which you want to see.
- Until means, the latest commit that you would like to see.
- We see reverse chronological order in Git and not normal chronological order.

git log --reverse : See the chronological order

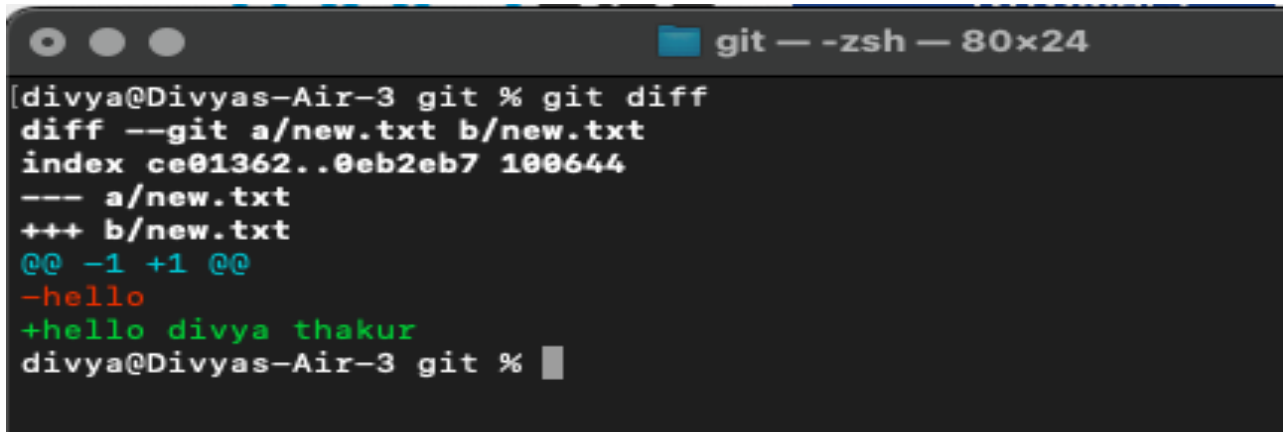
limited number of Git Commits History:

git log -n 2 -oneline : (issue last 2 git commits)

Git Diff Command

- Diff command is used in git to track the difference between the changes made on a file.
- Diff command takes two inputs and reflects the differences between them before commit.

Git Diff Command output :

A terminal window titled 'git - zsh - 80x24' showing the output of the 'git diff' command. The output includes file names 'a/new.txt' and 'b/new.txt', commit hashes 'ce01362' and '0eb2eb7', and a diff summary showing a change from '-hello' to '+hello divya thakur'.

```
[divya@Divyas-Air-3 git % git diff
diff --git a/new.txt b/new.txt
index ce01362..0eb2eb7 100644
--- a/new.txt
+++ b/new.txt
@@ -1,2 +1 @@
-hello
+hello divya thakur
divya@Divyas-Air-3 git %
```

The first line shows the file names that have been considered as the input in git diff. You can see that they have been marked by a and b along with the two different file state that has been taken as input.

This shows the metadata related to the command and the execution of it on the files.

This line defines the symbol, called a legend, to tell you what is used to describe the first file and what is used to describe the second file. As you can see, - is used in front of the first file, and + is used in front of the second file. So whenever diff shows you the changes related to the first file, they will be marked by - and the changes in the second file will be marked by the symbol +.

The fourth line shows you symbol @@ and symbols ahead of it. They are called chunks. Chunks in git diff define the change' summary. In our image the following chunk can be seen @@ -1,2 +1 @@

This means that lines one and two were changed in the first file and line one was changed in the second file. Remember the - and + symbol used in the third point as a symbol to the first and second file respectively.

Git Show command

- Git show is similar to git log ,the only difference is that with logs it shows :
 - Difference between the versions of the file to which HEAD is pointing (diff)
- Output:

```
[divya@Divyas-MacBook-Air-3 git % git show
commit de44cf935aa7daabb3c1ea3b0f0a660a7fed425b (HEAD -> master)
Author: Divya Thakur <divyathakurdt73@gmail.com>
Date: Tue Sep 17 00:36:55 2024 +0530

    committed

diff --git a/one.txt b/one.txt
index 86d447b..e69de29 100644
--- a/one.txt
+++ b/one.txt
@@ -1,0 @@
-hello,this is new data for file
divya@Divyas-MacBook-Air-3 git %
```

- git log --author=Divya : to see logs of specific author

Connecting a Git local repository with GitHub remote repository.

- A git remote command is used to make the remote connections such as connecting a Git local repository with GitHub remote repository.
- Git remote is just a connection between the local and GitHub repository.
- git remote can be considered as a reference to the GitHub repositories which do not provide any real-time access to what you do locally i.e. whatever you do locally will not be reflected on your GitHub repository without your permission.
- Git remote can be used to connect to your own repository or to connect to someone else's repository.

- git remote add name command :
- **git remote add origin https://github.com/Divyathakur273/devops.git**

The above command follows the generic syntax of **git remote add <name> <repository url>**

Add: To add a new URL to the repository.

Name: To give a name that you will use instead of the URL of the repository.

URL: The URL of the repository.

- To check whether we linked our repository or not, execute the **git remote** command. It will show you that the origin repository (alias for the GitHub myFirstRepo repository) is available.
- git remote -v command : show you the url of origin repo .
- git remote rm <name> : to remove a connection , in the above scenario name is origin.
- git remote rename <old-name> <new-name> : change name of old repo .

- git fetch : The git fetch command downloads commits, files, and refs from a remote repository into your local repo.

- Scenario :

1. git remote add new <https://github.com/Divyathakur273/devops.git>

2. git remote -v

3. git branch -r :to see branches

4. git fetch new : fetch the refs from new .

- git fetch new <branch name>

- git fetch --all

5. git branch -r

6. git log new/main: check the logs of all branches.

git fetch has similar behavior to git pull, however, git fetch can be considered a safer, nondestructive version.

Git fork

- A fork is a copy of a repository. Forking a repository allows to freely experiment with changes without affecting the original project.
- When a user forks a repository, all the files in the repository are automatically copied to the user's account on GitHub and it feels like the user's own repository.
- This process is similar to copying a folder from one drive to another drive on a computer.
- Through git forking, the users can develop their own modifications to the code that belongs to someone else.
- To be noted, that this process does not have any effect on the original repository (also called an upstream repository) code.
- Forking a repository on GitHub is done for two main purposes:
 - Improving someone's code/software
 - Reusing the code in a project
- Forking is allowed for public repositories without permission. But if the repository is private, one can only be able to fork if he or she has permission from the owner of the repository.



GITHUB.COM
CLOUD

CENTRAL REPOSITORY
(upstream)



FORK



PULL REQUEST



YOUR REPOSITORY



CLONE



LOCAL COMPUTER

PUSH



EDITS



EDITS



Cloning a Remote Repository from GitHub

- Cloning is a process of creating an identical copy of a Git Remote Repository to the local machine.
- A repository that is uploaded online for collaboration is called an Upstream Repository or a Central Repository.
- A central repository indicates that all the changes from all the contributors pushed into this repository only. So, this is the most updated repository instance of itself. Sometimes this is often called the original repository.



CENTRAL REPOSITORY
(upstream)



CLONE

PUSH

LOCAL COMPUTER



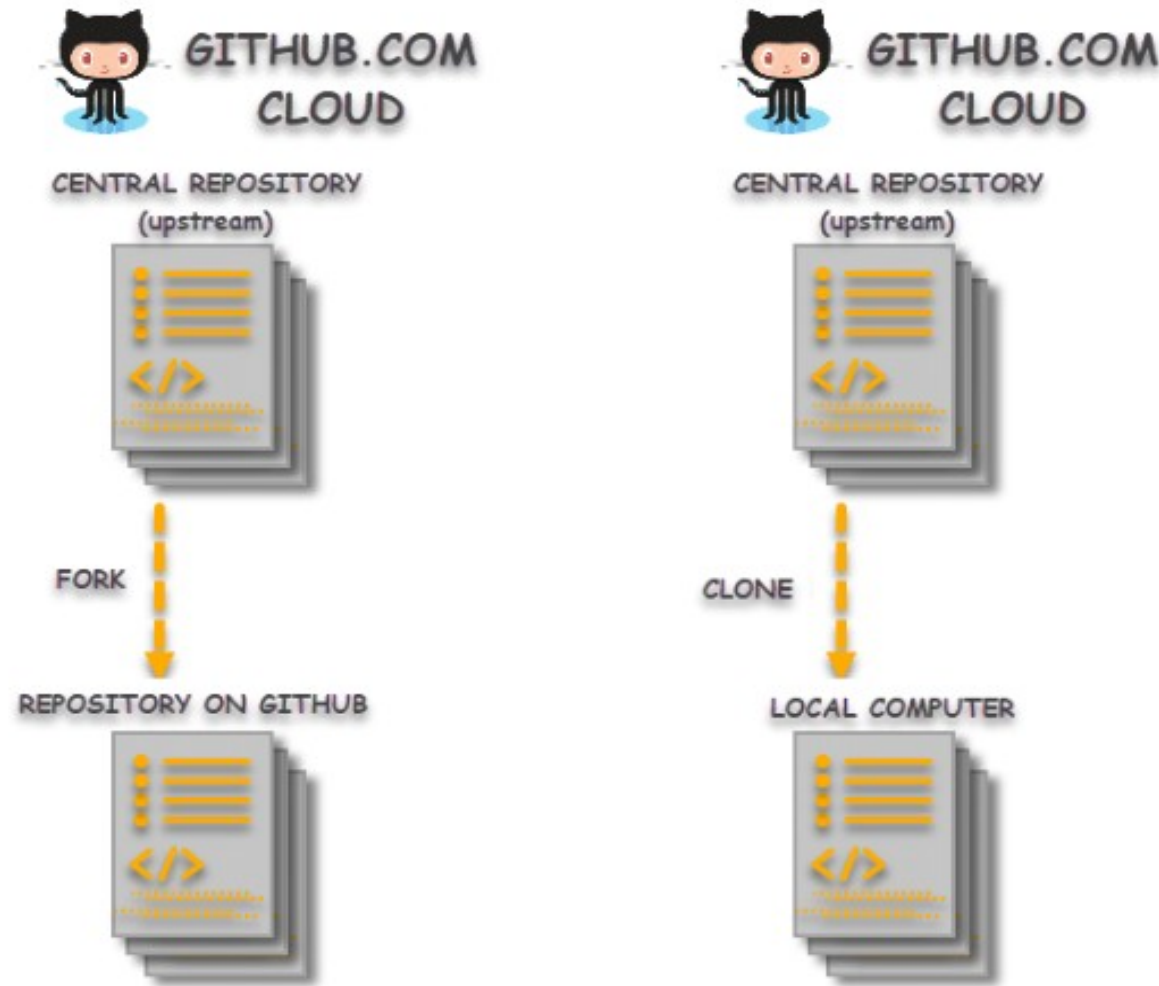
EDITS

EDITS



- `git clone <repoURL>`
 - It is a standard GIT command to clone an existing remote repository.

Difference btw git clone and git fork



git push

- A git push command, when executed, pushes the changes that the user has made on the local machine to the remote repository.
- To be able to push to your remote repository, you must ascertain that all your changes to the local repository are committed.
- Consider Git push as a part of the synchronization process in Git.
- The synchronization happens between the local and remote repository where the source and the receiver may vary.
- It uploads the changes done on the local repository to keep the remote repository up to date.
- command : git push



CENTRAL REPOSITORY
(upstream)



CLONE



LOCAL COMPUTER

PUSH



ADD



COMMIT



- command :
- `git push <remote_repo> <branch_name>`

Blob

- Git represents your file's contents in blobs, which are also leaf nodes in something close to a directory, called a tree.
- A Git blob (binary large object) is the object type used to store the contents of each file in a repository.
- The file's SHA-1 hash is computed and stored in the blob object.
- These endpoints allow you to read and write blob objects to your Git database on GitHub.
- Blobs leverage these custom media types.

- **git cat-file -p 21a40280c2ce1d4cdd0e6590f154479da9837495**
 - Here the command is git cat-file -p hashvalue
 - 21 is the folder inside which the hash is there
 - and a40280c2ce1d4cdd0e6590f154479da9837495 is the actual hash
 - to check the data of the file through hash
- **git cat-file -t 21a40280c2ce1d4cdd0e6590f154479da9837495**
 - Command to check the type of file ie, blob, tree etc

Step by step directions:

Step 1:

```
divya@Divyas-MacBook-Air-3 desktop % mkdir f1
divya@Divyas-MacBook-Air-3 desktop % cd f1
divya@Divyas-MacBook-Air-3 f1 % git init
Initialized empty Git repository in /Users/divya/Desktop/f1/.git/
divya@Divyas-MacBook-Air-3 f1 % ls -a
.  ..  .git
divya@Divyas-MacBook-Air-3 f1 % cd .git
divya@Divyas-MacBook-Air-3 .git % ls
HEAD      branches  config    description  hooks      info      objects    refs
divya@Divyas-MacBook-Air-3 .git % cd objects
divya@Divyas-MacBook-Air-3 objects % ls
info  pack
divya@Divyas-MacBook-Air-3 objects %
```

Step 2: Adding file to the staging area

```
divya@Divyas-MacBook-Air-3 f1 % ls
f1.txt
divya@Divyas-MacBook-Air-3 f1 % echo "this is the data of the file ">f1.txt
divya@Divyas-MacBook-Air-3 f1 % cat f1.txt
this is the data of the file
divya@Divyas-MacBook-Air-3 f1 % git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)

    f1.txt

nothing added to commit but untracked files present (use "git add" to track)
divya@Divyas-MacBook-Air-3 f1 % git add f1.txt
divya@Divyas-MacBook-Air-3 f1 % git status
On branch master

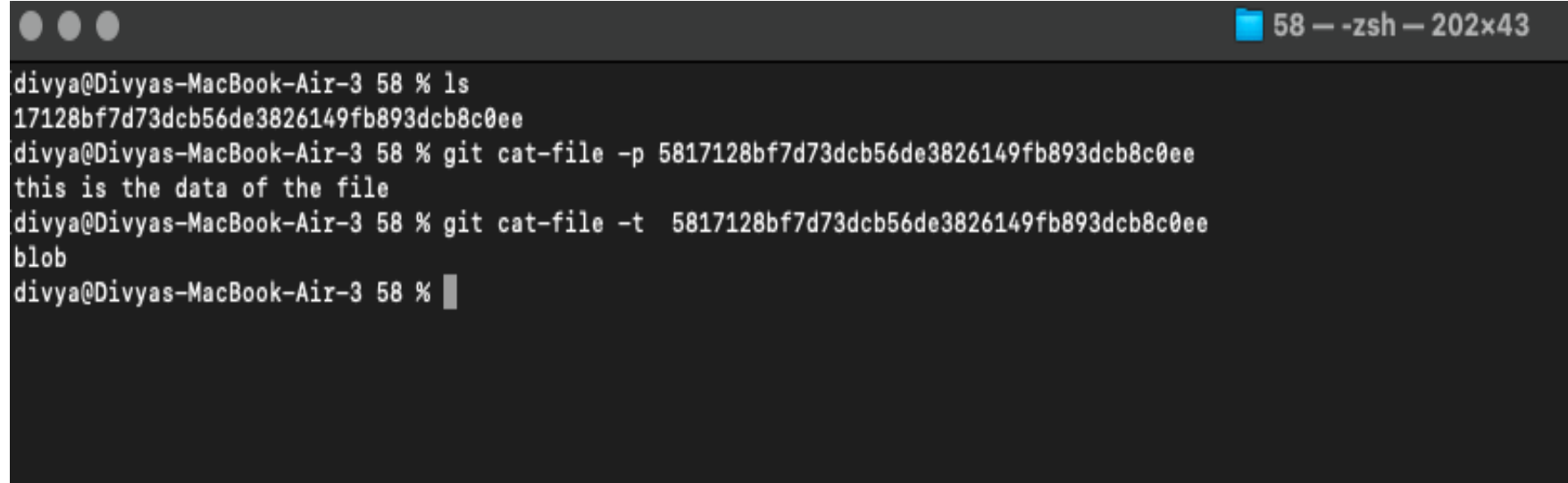
No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)

    new file:   f1.txt

divya@Divyas-MacBook-Air-3 f1 % cd ../git
cd: no such file or directory: ../git
divya@Divyas-MacBook-Air-3 f1 % cd .git
divya@Divyas-MacBook-Air-3 .git % cd objects
divya@Divyas-MacBook-Air-3 objects % ls
58      info      pack
divya@Divyas-MacBook-Air-3 objects % cd 58
divya@Divyas-MacBook-Air-3 58 %
```

Step 2:



```
divya@Divyas-MacBook-Air-3 58 % ls
17128bf7d73dcb56de3826149fb893dcb8c0ee
divya@Divyas-MacBook-Air-3 58 % git cat-file -p 5817128bf7d73dcb56de3826149fb893dcb8c0ee
this is the data of the file
divya@Divyas-MacBook-Air-3 58 % git cat-file -t 5817128bf7d73dcb56de3826149fb893dcb8c0ee
blob
divya@Divyas-MacBook-Air-3 58 %
```

Step 3 : from staging to commit

- Now , in step 2 , before commit , we have got one file in objects i.e, 58 , which contains hash value of the data inside the file f1.txt , which we created .
- We have only added the file to the staging area and not committed yet .
- In step 3 , we will add the file to the commit and then in the object folder , 2 new files will appear in which one file will contain hash value of commit and other will contain tree.

Step 3 : commit of file f1.txt

```
[divya@Divyas-MacBook-Air-3 58 % cd ..  
[divya@Divyas-MacBook-Air-3 objects % cd ..  
[divya@Divyas-MacBook-Air-3 .git % cd ..  
[divya@Divyas-MacBook-Air-3 f1 % git commit -m "committed"  
[master (root-commit) 2562b31] committed  
 1 file changed, 1 insertion(+)  
 create mode 100644 f1.txt  
[divya@Divyas-MacBook-Air-3 f1 % git status  
On branch master  
nothing to commit, working tree clean  
[divya@Divyas-MacBook-Air-3 f1 % cd .git  
[divya@Divyas-MacBook-Air-3 .git % cd objects  
[divya@Divyas-MacBook-Air-3 objects % ls  
25      58      d8      info      pack
```

After committing , new files created are 25,d8

```
d8 — -zsh — 202x43


```

Mktree and read tree

- Mktree : used to create a tree

`git cat-file | git mktree`

- Read tree: To read the contents of the tree.

`git read-tree <hashvalue>`